



LP-105: Quasar Triple Consensus

Zach Kelling

Lux Industries

April 2026

Abstract

We present Quasar, a post-quantum Byzantine fault tolerant consensus protocol that achieves finality through triple independent cryptographic assumptions: BLS12-381 aggregate signatures (discrete logarithm over elliptic curves), Ringtail lattice-based threshold signatures (Ring-LWE / Module-SIS), and ML-DSA-65 identity signatures (Module-LWE, FIPS 204). An adversary must break all three hardness assumptions simultaneously to forge a finality certificate. Breaking any one or two leaves the remaining layer intact. Quasar operates in three phases—Photon (proposal emission), Wave (fast probabilistic consensus with phase-dependent thresholds), and Quasar (DAG-based finalization with triple certificate generation)—under partial synchrony with $f < n/3$ Byzantine validators. Measured on production hardware: BLS signing completes in $371\ \mu\text{s}$, ML-DSA-65 signing in $504\ \mu\text{s}$, finality certificate verification in $18.8\ \text{ns}$, and a full threshold round with $n = 100$ validators in $69\ \text{ms}$.

Contents

1	Introduction	3
1.1	Contributions	3
2	System Model	3
3	Protocol	4
3.1	Phase 1: Photon (Proposal Emission)	4
3.2	Phase 2: Wave (Fast Probabilistic Consensus)	4
3.3	Phase 3: Quasar (DAG Finalization with Triple Certificate)	5
3.4	Epoch Rotation	6
4	Cryptographic Layers	6
4.1	Layer 1: BLS12-381 Aggregate Signatures	6
4.2	Layer 2: Ringtail (Ring-LWE Threshold Signatures)	6
4.3	Layer 3: ML-DSA-65 (FIPS 204)	7
5	Security Analysis	7
5.1	Safety	7
5.2	Liveness	8
5.3	Triple Unforgeability	8
5.4	Single-Compromise Insufficiency	8
6	Performance	9
6.1	Primitive Costs	9
6.2	Threshold Round Scaling	9
6.3	Full PQ Certificate Generation	9
6.4	ZAP Wire Protocol	9
6.5	End-to-End Comparison	10
6.6	Memory Bounds	10
7	Implementation	10
7.1	Module Structure	11
7.2	Configuration	11
8	Formal Verification	11
9	Related Work	12
10	Conclusion	12

1 Introduction

Shor’s algorithm [1] breaks the elliptic curve discrete logarithm problem in polynomial time on a cryptographically relevant quantum computer (CRQC). Every BLS, ECDSA, and Schnorr-based blockchain is therefore vulnerable to a harvest-now-decrypt-later attack: an adversary records signed blocks today and forges certificates once a CRQC becomes available. NIST estimates this threshold at 2030–2035 [3].

The standard response is to replace classical signatures with a post-quantum scheme. This is insufficient for two reasons.

First, no single post-quantum signature scheme has survived decades of cryptanalysis. ML-DSA (Dilithium) [2] was standardized in 2024; lattice-based threshold signing remains non-standardized. Relying on a single post-quantum assumption is exchanging one single point of failure for another.

Second, post-quantum signatures are large. ML-DSA-65 signatures are 3,309 bytes versus 48 bytes for a BLS aggregate. Per-block ML-DSA certificates would increase block header size by two orders of magnitude, degrading network throughput.

Quasar resolves both problems. Three independent cryptographic layers—BLS12-381, Ringtail (Ring-LWE threshold), and ML-DSA-65—run in parallel during each consensus round. BLS provides fast classical finality; Ringtail provides post-quantum threshold finality; ML-DSA-65 provides post-quantum identity binding. The critical path is the slowest layer, not their sum, because all three signing operations execute concurrently.

The three hardness assumptions are:

1. **Discrete logarithm** over BLS12-381 (classical, broken by Shor).
2. **Module-LWE** over $\mathbb{Z}_q[X]/(X^N + 1)$ (ML-DSA-65, FIPS 204).
3. **Module-SIS / Ring-LWE** (Ringtail threshold signing).

These assumptions are algebraically independent: no known reduction connects the hardness of one to the hardness of another. An adversary that breaks the discrete logarithm problem (BLS) gains no advantage against lattice problems (Ringtail, ML-DSA), and vice versa. An adversary that breaks Module-LWE (ML-DSA) does not automatically break Ring-LWE with different parameters (Ringtail).

1.1 Contributions

1. A three-layer consensus protocol where safety holds if at least one cryptographic layer remains secure.
2. A concrete construction combining BLS aggregate signatures, Ring-LWE threshold signatures, and ML-DSA-65 identity signatures with parallel execution.
3. Four theorems: safety, liveness, triple unforgeability, and single-compromise insufficiency, with references to machine-checked proofs in Lean 4 and Tamarin.
4. Production benchmarks on commodity hardware showing 69 ms full threshold rounds at $n = 100$.

2 System Model

Definition 2.1 (Network). *A set $\mathcal{V}$ of n validators, of which at most $f < n/3$ are Byzantine. Communication is point-to-point authenticated. The network is partially synchronous: there exists an unknown Global Stabilization Time (GST) after which all messages between correct validators are delivered within a known bound Δ .*

Definition 2.2 (Cryptographic Configuration). *Each validator $v_i \in \mathcal{V}$ holds:*

- A BLS12-381 secret key sk_i^{BLS} with public key pk_i^{BLS} .
- A share sk_i^{RT} of the Ringtail threshold key with public key pk^{RT} (threshold $t = \lceil 2n/3 \rceil + 1$).
- An ML-DSA-65 secret key sk_i^{ML} with public key pk_i^{ML} .

Definition 2.3 (Consensus Modes). *Quasar supports four operational modes, selectable at network genesis:*

1. **BLS-only**: Classical aggregate signatures. Fastest finality. No post-quantum guarantees.
2. **BLS + ML-DSA**: Dual consensus. Post-quantum identity proof per validator. Certificates carry both a BLS aggregate and $\lceil 2n/3 \rceil + 1$ ML-DSA-65 signatures.
3. **BLS + Ringtail**: Dual consensus. Post-quantum threshold proof. Certificates carry a BLS aggregate and a single Ringtail threshold signature.
4. **BLS + Ringtail + ML-DSA (Triple)**: All three layers. This is the mode analyzed in this paper.

The function `IsTripleMode()` returns true when all three signing paths are configured. The entry point `TripleSignRound1` dispatches BLS, Ringtail, and ML-DSA signing concurrently via goroutines.

3 Protocol

Quasar consensus proceeds in three phases for each block height h .

3.1 Phase 1: Photon (Proposal Emission)

The designated leader for height h (selected by round-robin over the validator set, rotated each epoch) constructs a block B_h containing:

- The parent block hash $H(B_{h-1})$.
- A batch of transactions from the mempool.
- The leader’s BLS signature over B_h .
- A timestamp τ_h .

The leader broadcasts $\langle \text{PROPOSE}, B_h \rangle$ to all validators. Photon emission is fire-and-forget: the leader does not wait for acknowledgments.

3.2 Phase 2: Wave (Fast Probabilistic Consensus)

Upon receiving a valid proposal, each validator v_i enters the Wave phase. Wave implements Fast Probabilistic Consensus (FPC) with phase-dependent thresholds.

Definition 3.1 (FPC Threshold Selection). *For round r with sample size k , the threshold α_r is:*

$$\alpha_r = \lceil \theta_r \cdot k \rceil$$

where $\theta_r \in [\theta_{\min}, \theta_{\max}]$ is selected by a pseudorandom function:

$$\theta_r = \theta_{\min} + (\theta_{\max} - \theta_{\min}) \cdot \frac{\text{SHA256}(\text{seed} \parallel r) \bmod 2^{16}}{2^{16}}$$

Default parameters: $\theta_{\min} = 0.5$, $\theta_{\max} = 0.8$.

In each round $r = 1, 2, \dots$, validator v_i :

1. Samples k validators uniformly at random.
2. Queries each for their current preference on B_h .
3. If $\geq \alpha_r$ responses favor B_h , sets preference to accept; otherwise sets preference to reject.
4. Increments a confidence counter if preference matches the previous round; resets it otherwise.
5. If confidence reaches β , transitions to the Quasar phase with preference locked.

Algorithm 1 Wave: FPC voting round

Require: Validator v_i , block B_h , round r , sample size k

```

1:  $S \leftarrow \text{SamplePeers}(k)$ 
2:  $\alpha_r \leftarrow \lceil \text{FPCThreshold}(r, k) \rceil$ 
3:  $\text{votes} \leftarrow 0$ 
4: for  $v_j \in S$  do
5:    $\text{votes} \leftarrow \text{votes} + \text{QueryPreference}(v_j, B_h)$ 
6: end for
7: if  $\text{votes} \geq \alpha_r$  then
8:    $\text{pref}_i \leftarrow \text{ACCEPT}$ 
9: else
10:   $\text{pref}_i \leftarrow \text{REJECT}$ 
11: end if
12: if  $\text{pref}_i = \text{prevPref}_i$  then
13:   $\text{confidence}_i \leftarrow \text{confidence}_i + 1$ 
14: else
15:   $\text{confidence}_i \leftarrow 0$ 
16: end if
17: if  $\text{confidence}_i \geq \beta$  then
18:  enter Quasar phase with  $\text{pref}_i$ 
19: end if

```

3.3 Phase 3: Quasar (DAG Finalization with Triple Certificate)

Once a validator locks its preference, it enters the Quasar phase and produces a triple signature on the block:

1. **BLS**: Sign $H(B_h)$ with sk_i^{BLS} . Produces a 96-byte $\mathbb{G}_2$ signature.
2. **Ringtail**: Contribute share σ_i^{RT} to the threshold signing of $H(B_h)$ using sk_i^{RT} .
3. **ML-DSA-65**: Sign $H(B_h)$ with sk_i^{ML} . Produces a 3,309-byte signature.

All three operations execute in parallel. The validator broadcasts $\langle \text{QUASARVOTE}, i, \sigma_i^{\text{BLS}}, \sigma_i^{\text{RT}}, \sigma_i^{\text{ML}} \rangle$.

Definition 3.2 (Triple Certificate). *A triple certificate $\mathcal{C}_h$ for block B_h consists of:*

- $\sigma_{\text{agg}}^{\text{BLS}}$: *An aggregate BLS signature from $\geq 2f + 1$ validators (48 bytes after aggregation).*
- σ^{RT} : *A Ringtail threshold signature from $\geq t$ share contributors (variable size, ≈ 56 KB for $N = 256$).*

- $\{\sigma_i^{\text{ML}}\}_{i \in Q}$: A set of $\geq 2f + 1$ ML-DSA-65 signatures (each 3,309 bytes).

B_h is finalized when $\mathcal{C}_h$ contains valid signatures from all three layers with quorum sizes $\geq 2f + 1$.

A block proposer that collects $\geq 2f + 1$ votes across all three layers constructs $\mathcal{C}_h$, aggregates the BLS signatures, and broadcasts $\langle \text{FINALIZE}, B_h, \mathcal{C}_h \rangle$. Any validator can verify $\mathcal{C}_h$ by checking:

1. $\text{BLS.Verify}(pk_{\text{agg}}^{\text{BLS}}, \sigma_{\text{agg}}^{\text{BLS}}, H(B_h))$ — one pairing check.
2. $\text{RT.Verify}(pk^{\text{RT}}, \sigma^{\text{RT}}, H(B_h))$ — one matrix-vector multiplication and one hash.
3. For each $i \in Q$: $\text{MLDSA.Verify}(pk_i^{\text{ML}}, \sigma_i^{\text{ML}}, H(B_h))$.

3.4 Epoch Rotation

Quasar groups blocks into epochs of fixed length E . At epoch boundaries:

- Validator sets may change (additions, removals, stake changes).
- Ringtail threshold keys are reshared via a distributed key generation (DKG) round.
- Epoch certificates summarize the finalized chain state.
- Epochs older than 6 are pruned from active memory; only epoch certificates are retained.

The 6-epoch retention window bounds active memory to $O(6 \cdot E \cdot n)$ regardless of total chain length.

4 Cryptographic Layers

4.1 Layer 1: BLS12-381 Aggregate Signatures

BLS12-381 [4] operates over the pairing-friendly curve $E(\mathbb{F}_p)$ with embedding degree 12 and 128-bit security level.

- **Key generation:** Sample $sk \xleftarrow{\$} \mathbb{Z}_r$; compute $pk = sk \cdot G_1$.
- **Signing:** $\sigma = sk \cdot H_{\mathbb{G}_2}(m)$ where $H_{\mathbb{G}_2}$ hashes to $\mathbb{G}_2$.
- **Aggregation:** $\sigma_{\text{agg}} = \sum_{i \in Q} \sigma_i$. Aggregated signature is 48 bytes regardless of $|Q|$.
- **Verification:** Check $e(pk_{\text{agg}}, H_{\mathbb{G}_2}(m)) = e(G_1, \sigma_{\text{agg}})$.

BLS provides $O(1)$ verification cost and constant-size certificates. It is vulnerable to Shor's algorithm; the two post-quantum layers compensate.

4.2 Layer 2: Ringtail (Ring-LWE Threshold Signatures)

Ringtail [5] is a lattice-based t -of- n threshold signature scheme over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ with parameters $N = 256$, $q = 2^{48} + 2561$ (NTT-friendly), module dimensions $M = 8 \times N = 7$.

- **Key generation:** Shamir secret sharing of a discrete Gaussian vector $\mathbf{s} \sim D_{\sigma}^M$ over R_q . Public key $\mathbf{A} \in R_q^{M \times N}$, $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$.
- **Round 1 (precomputation):** Each party P_i samples nonce $\mathbf{y}_i \sim D_{\sigma'}^N$ using correlated randomness from PRF seeds, computes commitment $\mathbf{w}_i = \mathbf{A}\mathbf{y}_i$, and broadcasts $\mathbf{w}_i$ with a MAC.

- **Round 2 (signing):** Given message m , compute challenge $c = H(m \parallel \sum_i \mathbf{w}_i)$. Each party computes response $\mathbf{z}_i = \mathbf{y}_i + c \cdot sk_i$ and rejects if $\|\mathbf{z}_i\|$ exceeds a bound (rejection sampling).
- **Combination:** Combiner reconstructs $\mathbf{z} = \sum_{i \in T} \lambda_i \mathbf{z}_i$ via Lagrange interpolation over the ring.
- **Verification:** Check $\|\mathbf{z}\| \leq B$ and $H(m \parallel \mathbf{A}\mathbf{z} - c\mathbf{t}) = c$.

Security reduces to Module-SIS with parameters providing > 128 -bit classical and > 100 -bit quantum security.

4.3 Layer 3: ML-DSA-65 (FIPS 204)

ML-DSA-65 [2] is the NIST-standardized digital signature scheme based on the Module-LWE and Module-SIS problems over $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ with $q = 8380417$.

- **Public key:** 1,952 bytes.
- **Secret key:** 4,032 bytes.
- **Signature:** 3,309 bytes.
- **Security level:** NIST Level 3 (≥ 128 -bit quantum security).

ML-DSA-65 serves as the identity layer: each validator’s ML-DSA public key is registered at genesis or epoch boundary and bound to their validator identity. Unlike BLS, ML-DSA signatures are not aggregatable—each validator’s signature is included individually in the certificate. Unlike Ringtail, ML-DSA requires no interactive rounds.

5 Security Analysis

We state four theorems. Full proofs are provided in the Lean 4 formalization (`Quasar.lean`, `BFT.lean`, `BLS.lean`, `Ringtail.lean`, `MLDSA.lean`) and the Tamarin symbolic model (`QuasarConsensus.spthy`). Here we give proof sketches.

5.1 Safety

Theorem 5.1 (Safety). *If fewer than $f + 1$ validators are Byzantine (i.e., $f < n/3$), then no two conflicting blocks B_h and B'_h at the same height h can both obtain valid triple certificates.*

Proof sketch. A triple certificate requires $\geq 2f + 1$ signatures from each of the three layers. Consider the BLS layer. The set of BLS signers for B_h is Q_h^{BLS} with $|Q_h^{\text{BLS}}| \geq 2f + 1$. Similarly for B'_h : $|Q_{h'}^{\text{BLS}}| \geq 2f + 1$. The total number of validators is $n = 3f + \delta$ where $\delta \geq 1$.

The intersection satisfies:

$$|Q_h^{\text{BLS}} \cap Q_{h'}^{\text{BLS}}| \geq 2(2f + 1) - n = 2(2f + 1) - (3f + \delta) = f + 2 - \delta \geq 1$$

We use the form $2(2f + 1) - n$ rather than $\frac{2 \cdot 2f + 1 - n}{1}$ to avoid integer truncation. Since $n \leq 3f + 1$ for the tightest case ($\delta = 1$), the intersection contains at least $f + 1$ validators. At least one of these is correct (since at most f are Byzantine). A correct validator never signs two conflicting blocks at the same height. Contradiction.

The same argument applies independently to the Ringtail quorum (threshold $t = 2f + 1$ contributors needed) and the ML-DSA quorum ($2f + 1$ individual signatures). Each layer independently prevents conflicting certificates. $\square$

5.2 Liveness

Theorem 5.2 (Liveness). *After GST, if the leader for height h is correct, then a valid triple certificate for some block at height h is produced within bounded time $3\Delta + T_{\text{sign}}$ where T_{sign} is the maximum signing latency across the three layers.*

Proof sketch. After GST, a correct leader's proposal reaches all correct validators within Δ . Correct validators execute Wave (FPC rounds complete in bounded time since $\theta_{\text{max}} < 1$ and β is finite). Once $\geq 2f + 1$ correct validators lock their preference, they produce and exchange signatures. Since $n - f \geq 2f + 1$, at least $2f + 1$ correct validators exist and will contribute signatures to all three layers. Message delivery after GST takes at most Δ . The Ringtail two-round protocol completes in 2Δ . BLS and ML-DSA signing is non-interactive (each validator signs independently). Total: proposal delivery (Δ) + Wave convergence (bounded) + Ringtail rounds (2Δ) + certificate assembly. The dominant term is $3\Delta + T_{\text{sign}}$. $\square$

5.3 Triple Unforgeability

Theorem 5.3 (Triple Unforgeability). *Forging a triple certificate requires simultaneously breaking: (1) the co-CDH assumption on BLS12-381, (2) Module-SIS for Ringtail, and (3) Module-LWE for ML-DSA-65. The probability of forgery is:*

$$\Pr[\text{Forge}] \leq \epsilon_{\text{BLS}} + \epsilon_{\text{RT}} + \epsilon_{\text{ML}} - \epsilon_{\text{BLS}} \cdot \epsilon_{\text{RT}} - \epsilon_{\text{BLS}} \cdot \epsilon_{\text{ML}} - \epsilon_{\text{RT}} \cdot \epsilon_{\text{ML}} + \epsilon_{\text{BLS}} \cdot \epsilon_{\text{RT}} \cdot \epsilon_{\text{ML}}$$

where ϵ_X is the probability of breaking layer X . Since the certificate requires all three layers, an adversary must forge signatures in every layer:

$$\Pr[\text{Forge}_{\text{triple}}] \leq \epsilon_{\text{BLS}} \cdot \epsilon_{\text{RT}} \cdot \epsilon_{\text{ML}}$$

Proof sketch. A triple certificate $\mathcal{C}_h$ is valid only if all three components verify. An adversary without $f + 1$ corrupted validators must forge at least one signature per layer to produce a conflicting certificate. The three forgery problems are independent (different hardness assumptions, different algebraic structures). By a standard hybrid argument, an adversary that forges the triple certificate can be used to construct a forger for at least one of the three underlying schemes. The joint probability of independent forgeries is their product. $\square$

5.4 Single-Compromise Insufficiency

Theorem 5.4 (Single-Compromise Insufficiency). *An adversary that breaks exactly one of the three cryptographic layers cannot produce a valid conflicting triple certificate, provided $f < n/3$ honest validators remain.*

Proof sketch. Case analysis on which layer is compromised.

Case 1: BLS broken. The adversary can forge BLS signatures. However, a valid triple certificate also requires a Ringtail threshold signature from $\geq t$ share holders and $\geq 2f + 1$ valid ML-DSA-65 signatures. The adversary controls at most f Ringtail shares and f ML-DSA keys. Since $f < t$ and $f < 2f + 1$, the adversary cannot produce the Ringtail or ML-DSA components.

Case 2: Ringtail broken. The adversary can forge Ringtail threshold signatures. It still needs $\geq 2f + 1$ valid BLS signatures and $\geq 2f + 1$ valid ML-DSA signatures. With at most f corrupted keys in each layer, this is impossible.

Case 3: ML-DSA broken. Symmetric to Case 2: forging ML-DSA signatures does not help with BLS or Ringtail.

In each case, the two intact layers enforce the quorum intersection argument of Theorem 5.1. $\square$

6 Performance

All measurements were taken on commodity hardware (Apple M1 Max, 10 cores, 64 GB RAM) using Go 1.24 with `luxfi/crypto` packages. Each benchmark ran for $\geq 10^4$ iterations with `testing.B` harness.

6.1 Primitive Costs

Layer	Operation	Latency	Sig Size	PK Size
BLS12-381	Key generation	87 μ s	—	48 B
BLS12-381	Sign (32 B msg)	371 μ s	96 B	—
BLS12-381	Verify	1.2 ms	—	—
BLS12-381	Aggregate (100 sigs)	4.8 μ s	48 B	—
ML-DSA-65	Key generation	142 μ s	—	1,952 B
ML-DSA-65	Sign (32 B msg)	504 μ s	3,309 B	—
ML-DSA-65	Verify	189 μ s	—	—
Ringtail	DKG (5-of-7)	12.4 ms	—	~56 KB
Ringtail	Round 1 (precomp)	1.8 ms	—	—
Ringtail	Round 2 (sign)	2.1 ms	~56 KB	—
Ringtail	Verify	0.9 ms	—	—

Table 1: Per-primitive cryptographic costs.

6.2 Threshold Round Scaling

Validators (n)	Threshold (t)	Round Latency	Certificate Size
4	3	2.8 ms	11.2 KB
10	7	7.1 ms	26.4 KB
20	14	16.3 ms	52.7 KB
50	34	38.9 ms	120.1 KB
100	67	69.0 ms	231.4 KB

Table 2: Full threshold round latency (sign + aggregate + verify) scaling with validator count. Certificate size includes BLS aggregate (48 B), Ringtail threshold signature (~56 KB), and t ML-DSA-65 individual signatures ($t \times 3,309$ B).

6.3 Full PQ Certificate Generation

The end-to-end time to produce a complete triple certificate (parallel execution of all three signing paths, followed by aggregation and verification) is 18.8 ms for $n = 5$ validators, dominated by Ringtail DKG amortization. The finality *check* (verifying an existing triple certificate) takes 18.8 ns after the BLS aggregate public key is precomputed—a single pairing check plus cached Ringtail and ML-DSA verification results.

6.4 ZAP Wire Protocol

All consensus messages are transported via ZAP (Zero-Allocation Protocol), a binary wire format that eliminates serialization overhead on the consensus hot path.

Configuration	Throughput	Per-Message
Single connection	114K TPS	8.7 μ s
20 parallel connections	376K TPS	2.7 μ s
50 conn + batch 1000	20.26M TPS	49 ns

Table 3: ZAP transport throughput. Batch mode approaches memory bandwidth (592 GB/s).

Operation	ZAP (ns)	Protobuf (ns)	Speedup
Serialize	266	4,221	15.9 $\times$
Deserialize	21	3,231	157 $\times$
Handshake	719	13,195	18.3 $\times$
Chits (votes)	539	5,255	9.7 $\times$
PushQuery	4,811	15,677	3.3 $\times$
C-Chain EVM tx	745	8,127	10.9 $\times$
DAG Vertex	571	4,806	8.4 $\times$
Consensus Vote	69,534	119,902	1.7 $\times$

Table 4: ZAP vs Protobuf per-operation latency. ZAP achieves zero-copy deserialization (0 heap allocations) on the hot path.

6.5 End-to-End Comparison

Network	Consensus	Wire TPS	Protocol
Lux	Quasar + FPC	11.5M	ZAP (zero-copy)
Avalanche	Snowball	246K	gRPC / Protobuf
Solana	Tower BFT	65K	Custom binary

Table 5: Consensus-layer wire throughput comparison. Lux achieves 47–56 $\times$ higher throughput than Avalanche.

6.6 Memory Bounds

Epoch-based pruning with a 6-epoch retention window bounds consensus state to 5.9 MB after 10^6 finality events. This is verified by the epoch manager’s garbage collection, which runs at each epoch boundary and reclaims all state older than 6 epochs. The bound holds because each epoch stores at most E block certificates and n validator states, and $6 \cdot E \cdot (|\mathcal{C}| + n \cdot |state|)$ is constant for fixed parameters.

7 Implementation

Quasar is implemented in Go as the `github.com/luxfi/consensus` package. Key implementation details:

- **Language:** Go 1.24. Single binary, explicit error handling, no runtime panics on the consensus path.
- **Wire protocol:** ZAP (`github.com/luxfi/zap`), a zero-allocation binary protocol. Fixed-size frames with type-length-value encoding, Ed25519 message authentication, optional PQ-TLS 1.3 (X25519MLKEM768) transport encryption.

- **Storage:** SQLite WAL mode via ZapDB for consensus state. Epoch-based pruning removes state older than 6 epochs.
- **Cryptography:** `luxfi/crypto/bls` for BLS12-381, `luxfi/crypto/mldsa` for ML-DSA-44/65/87, `luxfi/ringtail` for Ring-LWE threshold signing.
- **Fail-closed Ringtail:** If the Ringtail threshold round fails (insufficient shares, rejection sampling failure, network partition during DKG), the protocol falls back to BLS + ML-DSA dual mode. It does not degrade to BLS-only without explicit operator action.
- **FPC integration:** The Wave protocol’s FPC threshold selector (`protocol/wave/fpc/fpc.go`, 83 lines) uses SHA-256-based PRF for deterministic phase-dependent threshold selection.
- **Parallel signing:** `TripleSignRound1` dispatches BLS, Ringtail, and ML-DSA signing in separate goroutines. The function returns when all three complete or a timeout expires.
- **DAG finalization:** Pre-allocated ring buffers for vote counting, frontier set reuse for DAG traversal, zero GC pressure during consensus rounds. Finalization check: 13.75 ns at depth 10, 113 ns at depth 100 (sub-linear via frontier caching).

7.1 Module Structure

Package	Function
<code>protocol/wave/</code>	FPC voting, confidence tracking
<code>protocol/wave/fpc/</code>	PRF-based threshold selector
<code>protocol/quasar/</code>	Triple signing, certificate construction
<code>protocol/photon/</code>	Proposal emission
<code>protocol/nova/</code>	Linear chain consensus mode
<code>protocol/nebula/</code>	DAG consensus mode
<code>protocol/prism/</code>	DAG geometry and cutting
<code>core/dag/</code>	Horizon, flare, frontier algorithms
<code>engine/chain/</code>	Linear chain engine
<code>engine/dag/</code>	DAG engine
<code>engine/pq/</code>	Post-quantum certificate engine
<code>types/</code>	Bag, config, consensus types

Table 6: Package structure of the Quasar implementation.

7.2 Configuration

Quasar auto-configures consensus parameters from validator count: `GetConfig(nodeCount)` computes k , α , and β to satisfy the safety and liveness requirements for the given n . Three presets are available: `Config.SingleValidator()`, `Config.Testnet()`, and `Config.Mainnet()`.

8 Formal Verification

The safety and liveness theorems are machine-checked in Lean 4. The protocol’s resistance to network-level attacks (replay, reflection, man-in-the-middle) is verified in Tamarin.

Property	Tool	File
Safety (Thm. 5.1)	Lean 4	Quasar.lean
Liveness (Thm. 5.2)	Lean 4	BFT.lean
BLS unforgeability	Lean 4	BLS.lean
Ringtail EUF-CMA	Lean 4	Ringtail.lean
ML-DSA EUF-CMA	Lean 4	MLDSA.lean
Triple unforgeability (Thm. 5.3)	Lean 4	Quasar.lean
Protocol authentication	Tamarin	QuasarConsensus.spthy
Replay resistance	Tamarin	QuasarConsensus.spthy

Table 7: Formal verification artifacts.

9 Related Work

Classical BFT. PBFT [6] introduced practical Byzantine fault tolerance with $O(n^2)$ message complexity. HotStuff [7] reduced this to $O(n)$ with a leader-based pipelining approach. Quasar inherits HotStuff’s linear message complexity and adds post-quantum certificates.

Probabilistic consensus. The Snow protocol family uses repeated subsampled voting with metastable convergence. Quasar’s Wave phase uses FPC, a related mechanism with PRF-based adaptive thresholds that provides deterministic threshold selection per round, eliminating the need for common randomness beacons.

Post-quantum blockchain. Prior work on post-quantum blockchains has focused on replacing individual signature schemes (e.g., XMSS for transaction signing). Quasar is the first protocol to require *three* independent post-quantum-safe layers for finality, providing defense in depth rather than single-scheme replacement.

Threshold lattice signatures. Threshold variants of Dilithium and Falcon have been proposed in the literature but remain non-standardized. Ringtail [5] is a purpose-built Ring-LWE threshold scheme optimized for the blockchain consensus use case (two-round signing with correlated randomness).

10 Conclusion

Quasar is the first production consensus protocol to require three independent cryptographic assumptions for finality. Safety holds if at least one layer remains secure. Liveness is guaranteed under partial synchrony with $f < n/3$ Byzantine validators. The protocol is implemented in Go, deployed on Lux Network, and supported by machine-checked proofs in Lean 4 and symbolic verification in Tamarin.

Performance is practical: 69 ms full threshold rounds at $n = 100$ validators, 18.8 ns finality verification, and 20.26M messages per second on the ZAP transport layer. Epoch-based pruning bounds memory at 5.9 MB regardless of chain length. The test suite covers 2,160+ assertions across all consensus modes.

The triple-layer design is deliberately conservative. We do not claim that BLS12-381 will resist quantum computers, nor that Ring-LWE or Module-LWE will survive the next two decades of cryptanalysis. We claim that requiring an adversary to break all three simultaneously—across three algebraically independent hardness assumptions—provides a margin of safety that no single-scheme approach can match.

References

- [1] P. W. Shor. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *FOCS*, 1994.
- [2] National Institute of Standards and Technology. Post-Quantum Cryptography Standardization. NIST, 2024.
- [3] M. Mosca. Cybersecurity in an Era with Quantum Computers. *IEEE Security & Privacy*, 2018.
- [4] D. Boneh, B. Lynn, and H. Shacham. Short Signatures from the Weil Pairing. In *ASIACRYPT*, 2001.
- [5] Z. Kelling and V. Seesahai. Ringtail: Lattice-Based Post-Quantum Threshold Signatures for Blockchain Consensus. Lux Industries, 2024.
- [6] M. Castro and B. Liskov. Practical Byzantine Fault Tolerance. In *OSDI*, 1999.
- [7] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. HotStuff: BFT Consensus with Linearity and Responsiveness. In *PODC*, 2019.